Theory of Computation Scribe: Complexity Classes P, NP, and PSPACE

Scribe Team:
Aditya Mishra (2022MT11271)
Kamal Nehra (2022MT61979)
Subham Kumar (2022MT11823)
Dhruv Chaurasiya (2022MT111172)
Ayush Deep (2022MT61242)

Lecture Date: November 3, 2023

1 Introduction

This scribe details the lecture on time and space complexity. We begin by defining the standard deterministic and non-deterministic time and space complexity classes. We then explore the relationships between these classes, introducing the major classes **P**, **NP**, **PSPACE**, and **EXPTIME**.

A key result, **Savitch's Theorem**, is presented, which establishes that $PSPACE = NPSPACE$. Finally, we introduce the concepts of polynomial-time reductions, **NP-hardness**, and **NP-completeness**, which are central to the most famous open question in computer science: $P \stackrel{?}{=} NP$.

2 Basic Complexity Classes

The central idea is to classify languages based on the resources (time or space) a Turing Machine (TM) needs to decide them.

Definition 2.1 (Time Complexity)**.** Let M be a deterministic Turing Machine (DTM) that halts on all inputs. The **time complexity** of M is the function $f : \mathbb{N} \to \mathbb{R}^+$, where $f(n)$ is the maximum number of steps M uses on any input of length n.

We define the **time complexity class** $TIME(t(n))$ as:

$$TIME(t(n)) = \{L \mid L \text{ is a language decided by an } O(t(n)) \text{ time DTM}\}$$

Definition 2.2 (Space Complexity)**.** Let M be a DTM that halts on all inputs. The **space complexity** of M is the function $f : \mathbb{N} \to \mathbb{R}^+$, where $f(n)$ is the maximum number of tape cells M scans on any input of length n.

We define the **space complexity class** $SPACE(f(n))$ as:

$$SPACE(f(n)) = \{L \mid L \text{ is a language decided by an } O(f(n)) \text{ space DTM}\}$$

We define the non-deterministic classes **NTIME** and **NSPACE** analogously, based on the resources used by a non-deterministic Turing Machine (NTM) on any computation branch.

3 Relationships Between Basic Classes

From the definitions, we can derive several fundamental relationships.

[Basic Inclusions] Let $t(n)$ be a time complexity function and $f(n)$ be a space complexity function.

1. $TIME(t(n)) \subseteq NTIME(t(n))$
2. $SPACE(f(n)) \subseteq NSPACE(f(n))$
3. $TIME(t(n)) \subseteq SPACE(t(n))$
4. $NTIME(t(n)) \subseteq SPACE(t(n))$

Proof. (1) and (2) are true because a deterministic TM is a special case of a non-deterministic TM. (3) is true because a TM that runs for $t(n)$ steps can, at most, visit $t(n)$ new tape cells. (4) is true for the same reason, as the time complexity of an NTM is measured by its longest branch. □

The relationship between deterministic and non-deterministic time is widely believed to be exponential.

Theorem 3.1 (Deterministic vs. Non-deterministic Time)**.** *Every $t(n)$ time non-deterministic Turing machine has an equivalent $2^{O(t(n))}$ time deterministic Turing machine.*

$$NTIME(t(n)) \subseteq TIME(2^{O(t(n))})$$

Proof. (Sketch) A DTM can simulate an NTM by performing a breadth-first search of the NTM's computation tree. If the NTM runs in $t(n)$ time, the computation tree has a depth of at most $t(n)$. If the NTM has a branching factor of b, the total number of nodes in the tree is $O(b^{t(n)})$. The DTM simulation visits each node, resulting in an exponential time cost. □

In contrast, the relationship between deterministic and non-deterministic space is only polynomial. This is a foundational and non-intuitive result.

Theorem 3.2 (Savitch's Theorem)**.** *For any function $f : \mathbb{N} \to \mathbb{R}^+$ where $f(n) \geq \log n$:*

$$NSPACE(f(n)) \subseteq SPACE(f(n)^2)$$

Proof. (Sketch) We want to simulate an $NSPACE(f(n))$ machine N with a $SPACE(f(n)^2)$ machine D. The proof relies on a recursive algorithm, $CAN\_YIELD(c_1, c_2, t)$, which tests if N can get from configuration c_1 to c_2 in at most t steps.

1. To solve $CAN\_YIELD(c_1, c_2, t)$, the algorithm iterates through all possible intermediate configurations c_m.
2. For each c_m, it recursively calls $CAN\_YIELD(c_1, c_m, t/2)$ and $CAN\_YIELD(c_m, c_2, t/2)$.

3. If both recursive calls return true, then c_1 can reach c_2.

A configuration of N on an input of length n uses $O(f(n))$ space. The total number of steps t can be exponential, specifically $2^{O(f(n))}$. However, the recursion depth is only $\log t = \log(2^{O(f(n))}) = O(f(n))$.

Since each stack frame in the recursion stores a configuration (which takes $O(f(n))$ space), and the depth of the recursion is $O(f(n))$, the total space used by the deterministic simulation is $O(f(n)) \times O(f(n)) = O(f(n)^2)$. □

4 The Major Complexity Classes

We now define the most common complexity classes, which are based on polynomial resource bounds.

Definition 4.1 (Class **P**)**. P** is the class of languages that are decidable in polynomial time on a deterministic Turing machine.

$$P = \bigcup_{k \geq 0} TIME(n^k)$$

Definition 4.2 (Class **NP**)**. NP** is the class of languages that are decidable in polynomial time on a non-deterministic Turing machine.

$$NP = \bigcup_{k \geq 0} NTIME(n^k)$$

Definition 4.3 (Class **PSPACE**)**. PSPACE** is the class of languages that are decidable in polynomial space on a deterministic Turing machine.

$$PSPACE = \bigcup_{k \geq 0} SPACE(n^k)$$

Definition 4.4 (Class **NPSPACE**)**. NPSPACE** is the class of languages that are decidable in polynomial space on a non-deterministic Turing machine.

$$NPSPACE = \bigcup_{k \geq 0} NSPACE(n^k)$$

Definition 4.5 (Class **EXPTIME**)**. EXPTIME** (or **EXP**) is the class of languages that are decidable in exponential time on a deterministic Turing machine.

$$EXPTIME = \bigcup_{k \geq 0} TIME(2^{n^k})$$

5 Relationships Between the Major Classes

Based on the theorems from the previous section, we can establish the relationships between these major classes.

Theorem 5.1 (The "Big Picture")**.** *The relationships between the classes are as follows:*

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

Proof.
- $P \subseteq NP$: Follows from $TIME(n^k) \subseteq NTIME(n^k)$. A DTM is a special case of an NTM.
- $NP \subseteq PSPACE$: Follows from $NTIME(t(n)) \subseteq SPACE(t(n))$. A machine running in polynomial time n^k can only use polynomial space n^k. Therefore, $NTIME(n^k) \subseteq SPACE(n^k)$. Taking the union over all k gives $NP \subseteq PSPACE$.
- $PSPACE \subseteq EXPTIME$: Follows from $SPACE(f(n)) \subseteq TIME(2^{O(f(n))})$. If a language is in $PSPACE$, it is in $SPACE(n^k)$ for some k. This is contained in $TIME(2^{O(n^k)})$, which by definition is contained in $EXPTIME$.

□

We can also apply Savitch's Theorem to the polynomial classes.

Theorem 5.2. $PSPACE = NPSPACE$

Proof. We know $PSPACE \subseteq NPSPACE$ trivially. For the other direction, Savitch's Theorem states $NSPACE(f(n)) \subseteq SPACE(f(n)^2)$. If a language is in $NPSPACE$, it is in $NSPACE(n^k)$ for some k. By Savitch's Theorem, $NSPACE(n^k) \subseteq SPACE((n^k)^2) = SPACE(n^{2k})$. Since $SPACE(n^{2k})$ is contained in $PSPACE$, we have $NPSPACE \subseteq PSPACE$. Therefore, $PSPACE = NPSPACE$. □

We also have hierarchy theorems that prove that deterministic space and time classes are strictly contained within larger ones.

Theorem 5.3 (Space Hierarchy Theorem)**.** *For any space-constructible function $f(n)$, $SPACE(o(f(n))) \subsetneq SPACE(f(n))$.*

A direct corollary of this theorem is that $PSPACE \subsetneq EXPTIME$.

Corollary 5.4. $PSPACE \neq EXPTIME$

Proof. $PSPACE = \bigcup_k SPACE(n^k)$. This class is strictly contained in $SPACE(2^n)$, which is a subset of $EXPTIME$. □

This gives us the following chain of relationships, which is a cornerstone of complexity theory:

$$P \subseteq NP \subseteq PSPACE \subsetneq EXPTIME$$

This proves that at least one of the inclusions $P \subseteq NP$ or $NP \subseteq PSPACE$ must be a proper subset, but we do not know which. The most famous open problem in computer science is whether $P = NP$.

6 NP-Completeness

To understand the $P \stackrel{?}{=} NP$ problem, we study the "hardest" problems in NP. This requires the concept of a polynomial-time reduction.

Definition 6.1 (Polynomial Time Reduction)**.** A language A is **polynomial time reducible** to a language B, written $A \leq_p B$, if there is a polynomial-time computable function $f : \Sigma^* \to \Sigma^*$ such that for every w:

$$w \in A \iff f(w) \in B$$

The function f is called a polynomial-time reduction.

If $A \leq_p B$, it means that B is "at least as hard as" A. We can use this to classify problems.

Theorem 6.2. *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Proof. To decide A, first compute $f(w)$ in polynomial time, and then run the polynomial-time decider for B on the result $f(w)$. The composition of two polynomial-time algorithms is also polynomial-time. □

Theorem 6.3 (Transitivity of Reductions)**.** *The $\leq_p$ relation is transitive. That is, if $A \leq_p B$ and $B \leq_p C$, then $A \leq_p C$.*

These reductions allow us to define the hardest problems in NP.

Definition 6.4 (NP-hard)**.** A language L is **NP-hard** if every language $A \in NP$ is polynomial-time reducible to L. (i.e., $\forall A \in NP, A \leq_p L$).

Definition 6.5 (NP-complete)**.** A language L is **NP-complete** if:

1. $L \in NP$, and
2. L is NP-hard.

The NP-complete languages are the hardest problems in NP.

Corollary 6.6. *If any NP-complete language L is in P, then $P = NP$.*

This is the most practical consequence of the theory. If you can find a polynomial-time algorithm for even one NP-complete problem (like SAT, Hamiltonian Path, or Vertex Cover), you have proven that $P = NP$. Conversely, if $P \neq NP$, then no NP-complete problem can be solved in polynomial time.

7 Example: Primality Testing

An interesting example of a problem whose classification was a long-standing question is ***PRIMES****.*

$$PRIMES = \{\langle p \rangle \mid p \text{ is a prime number}\}$$

It was known for a long time that $PRIMES \in NP$ (and also $PRIMES \in coNP$). However, it was unknown if $PRIMES \in P$.

*In 2002, Manindra Agrawal, Neeraj Kayal, and Nitin Saxena (AKS) published a paper, "PRIMES is in P" [**?**]. They presented a deterministic, polynomial-time algorithm for primality testing. This was a major breakthrough, but it did not resolve the $P \stackrel{?}{=} NP$ question, as it is not known if $PRIMES$ is NP-complete.*